

07-开源贡献指南（v3）

开源贡献指南（v3：双主社区信任线，与 P1/P2 步骤耦合）

2026-10-05 v3 重构：依据 04/05 v3 重构与 2026-10-05 社区现勘重写，取代 v2（2026-10-04，B-lite）。配套：04（P1）、05（P2）、06（S6 调度插件）。贡献不是孤立动作，是做项目时”踩坑与测量的自然出口”。

1. 目标与验收（截止日驱动，不按 PR 数量自我感动）

贡献动机不变：第三方验证 + 面试素材 + 真实协作训练（简历行是副产品）。

deadline	最低交付（信任线）	冲刺交付
2027.2 实习投递	Fluid ≥1 个带测量数据的 issue（维护者正式回应/分类）+ 流程级 PR 1 个（docs/test，merged 或进入 review）	Fluid 代码 PR 在途
2027.8 秋招	AlBrix ≥1 个代码级 merged PR（或维护者认可方向、PR 在 review 的完整证据链）+ 两社区合计 ≥3 个 merged PR/已回应 issue	AlBrix ≥2 代码 PR；Fluid 成员提名对话

指标定义（v3 核心修正）：

- 信任线 = 在同一个社区内 docs/test → code 的连续记录，讲的是”维护者愿意再 review 你的东西”；“横跨 N 个社区”是虚荣指标，各撒一铲不如一条信任线；
- 每个 PR/issue 必须能讲满”发现 → 定位 → 取舍 → review 攻防”一个完整故事，否则不计入展示；
- typo/格式类 PR 不展示（但可作为第一次走流程的合法动作，限 1 次）；被关掉的 PR 不隐藏——“为什么被关、我学到了什么”同样是素材。

2. 社区情报档案（2026-10-05 现勘，入坑前以仓库当日状态复核）

社区	仓库/许可	法律门槛	语言/渠道	入口标签	与我的关系与定位
Fluid ★P1 主社区	fluid-cloudnative/fluid，Apache-2.0，CNCF 沙箱	DCO（git commit -s），无 CLA	中英可，钉钉群活跃；开发指南 docs/en/dev/how_to_develop.md	issue 模板分类；PR 模板	P1 闭环叙事同域（Dataset/DataLoad/调度注入）；主回馈目标

社区	仓库/许可	法律门槛	语言/渠道	入口标签	与我的关系与定位
AlBrix ★ P2 主社区	vllm-project/aibrix, Apache-2.0	DCO/CLA 以首 PR 时 CI 机器人为准 (vllm-project 系通常 DCO), 阶段 0 核实	英文为主, Slack: vllm-dev #AIBrix; 中文沟通可行	good first issue / help wanted	P2 核心 2 (冷启动协同) 的回馈目标; 代码 PR 主战场
JuiceFS CE	juicedata/juicefs, Apache-2.0 (旧"AGPLv3"说法已作废)	必须签 CLA (cla-assistant, 首 PR 时引导); 新源文件需 license header; pre-commit 必装	中文完全可, Discord/GitHub	kind/good-first-issue / kind/help-wanted	P1 对照基线; 使用与观察为主 , 只接"使用中自然踩到"的 docs/小 bug PR; 缓存组等 EE 能力不向 CE 提
vLLM Production Stack	vllm-project/production-stack, Apache-2.0	DCO	英文	good first issue	P2 对照对象; Go router 同域, 作为 AlBrix 之外的备选观察
llm-d	CNCF 沙箱 (2026 年新)	DCO	英文	—	P2 仅读码观察, 不投入贡献
scheduler-plugins	kubernetes-sigs/scheduler-plugins, Apache-2.0	CNCF DCO	英文, K8s Slack	help wanted	S6 流程练习场 : 练 fork/PR/CI 用, 不背名额、不与主社区抢时间
Dragonfly	CNCF, Apache-2.0	DCO	中文可	good first issue	仅当 P1 P2P 臂做深时才激活, 否则不碰
vLLM 引擎本体	vllm-project/vllm, Apache-2.0	DCO	英文	good first issue	不做贡献目标 : Python/CUDA 域与语言栈错位, 只走读代码

AlBrix 维护者分工 (@ 人时按域, 别乱 @): routing/gateway = @varungup90、@xunzhuo; autoscaling = @nwangfw; KV offload/分离 = @DwyaneShi; benchmark/研究合作 = @happyandslow; control plane 稳定性 = @googs1025。组织成员路径: 先有若干贡献 → 发 issue 申请加入 GitHub org。

法律红线: ① 自己的仓库 (TrainCache/KVRouter) 用 **Apache-2.0**; ② 任何社区代码可以读、可以学设计, **绝不拷贝进自己仓库** (即使同为 Apache-2.0, 也要独立实现并在 ADR 里注明"参考了 X 的思路"); ③ JuiceFS 企业版代码/文档是专有资产, 只能引用行为结论, 禁止搬运; ④ 首次向任何社区提交前, 把该仓库 CLA/DCO 的实际 CI 检查跑通一次 (故意在测试分支上验证红字长什么样, 不算浪费时间)。

3. 三阶段检查点（与 04/05 步骤精确耦合，不提前许愿）

检查点	时点（硬锚点）	决定什么	依据
CP0 = 现在	本周（使用阶段启动）	stage0 立刻并行开跑，不等待可选项	纯学习动作、零沉没成本
CP1	P1 步骤 0 现勘报告完成（约 2 周内）	Fluid 首个 issue 的选题（文档不符/行为缺口/可观测性）	现勘取证结果，不靠想象
CP2	P1 步骤 7（核心 1 闭环）跑通后	Fluid 代码 PR 的选题与形态（先 issue 讨论）	盈亏曲面里哪个发现对上游真有用
CP3	P2 阶段 0 现勘闸门关闭后	AlBrix 首个互动议题（冷启动测量 / 精准vs估算复现）	头号风险（CPU KV 事件）结论 + 读码笔记
CP4	P2 核心 2 主实验后	AlBrix 代码 PR（routing policy/选项/benchmark）	旗舰图 2 的数据与维护者反馈
逃生舱	任何时点	降级为”只提 issue/数据，不写 PR”，简历叙事转回项目为主	流程长期受阻且吞噬项目工期时启用

主社区是**被项目进度推出来的**：Fluid 由 P1 现勘与核心 1 推出，AlBrix 由 P2 现勘与核心 2 推出。不存在”今天选错”，只存在”还没走到证据”。

4. 阶段 0：观察者（本周启动，与双现勘合并执行）

减负原则（v3 关键修正）：不在两个全家桶上各交一遍全量环境税。

Fluid（全量做，约 3–4 个晚上）

1. `watch fluid-cloudnative/fluid`（All activity）；star 不算动作；
2. fork + clone，配 upstream remote（官方 CONTRIBUTING 的 5 步照做），`make test` 或 dev guide 指定命令跑通相关 Go 包单测——Fluid 是纯 Go 仓，笔记本可跑，跑不通的包记录原因（这可能就是 issue 素材）；
3. **精读 10 个近期 merged PR**，每篇结构化笔记四行：`diff 规模 / commit 怎么切 / reviewer 追问什么 / 我能学到的形状`；
4. 精读 `CONTRIBUTING.md` + PR/issue 模板 + dev guide，记住：DCO、PR 模板、每人最多 15 个 open PR、至少 1 个 approval 才 merge；
5. 钉钉/Slack 潜水，记录社区响应速度（这个数据决定 CP1 信心）。

AlBrix（轻量做，约 2–3 个晚上，拒绝全家桶黑洞）

1. `watch vllm-project/aibrix`；加入 vllm-dev Slack `#AIBrix`；
2. **只跑 quickstart**（helm 部署 + 一个示例请求打通），不要求本地编译 Envoy/Redis 全家桶、不要求全测试绿；

3. 读 docs/ 的贡献页与 pkg/ 中 gateway/routing/autoscaling 相关包，**定向跑这些包的 Go 单测**；首 PR 需要时再补完整开发环境；
4. 同样精读 **10 个 merged PR**（四行笔记），重点看 2026 年高频改动的 routing/connector/autoscaling 目录——顺便完成 P2 阶段 0 的读码任务，一份投入两份产出；
5. 核实 CI 的 DCO/CLA 实际形态、PR 标题惯例、lint 命令，记入笔记。

stage0 完成标准：Fluid 相关包测试可跑 + 两社区各 10 篇 PR 笔记 + AIBrix quickstart 截图 + 一页《社区地形笔记》（响应速度、常见 reviewer、PR 典型形状、禁忌）。**时间盒 1.5 周，超时先保 Fluid 全量与 AIBrix quickstart。**

5. 阶段 1：issue 报告者（P1/P2 现勘顺产，零额外选题成本）

现勘即贡献——这是 v3 最重要的操作心法：你按 04/05 做现勘时产出的每一个”文档与实际行为不符”，都是一个低风险、高可信度的 issue。

候选选题（从证据里挑，不允许凭空想）：

- **Fluid 方向（CP1）：**① 现勘实测 webhook 注入的 nodeAffinity 行为与文档描述不一致/文档未写清权重覆盖条件 → docs bug issue；② DataLoad 与调度时序的可观测性缺口（无事件/指标表明预热进度）→ feature proposal issue，**附你的实测时序证据**；③ kind/特定版本下部署文档的坑（复现步骤齐全）；
- **AIBrix 方向（CP3）：**① 冷启动测量话题——先开 discussion/issue 描述你的复现计划（扩缩容事件 TTFT 凹陷测量方法），问社区是否已知/是否有相关 WIP；② 精准 vs 估算两种模式在 CPU 后端的行为差异/文档空白；③ quickstart/文档中你踩过具体坑。

纪律：先搜 issue/PR/discussion 历史（含 closed）；中文社区可用中文写，AIBrix 用英文（AI 只准润色，见 §9）；1-2 个即可，不刷数。**完成标准：**≥1 个 issue 得到 maintainer 正式回应（回复/分类/标 label/转到 discussion）。

6. 阶段 2：流程练习 PR（各社区 1 个封顶）

- **Fluid：**现勘中发现的文档错漏/过时部署说明的 docs PR——**这是风险最低的第一个 PR**，走完整流程：fork → branch → `git commit -s` → PR 模板填全（动机/改动/验证）→ CI → 回应 review；
- **AIBrix：**给你读过的 routing/autoscaling 包补一个**边缘场景测试**（如副本重启/端点摘除后的路由状态），选题先在对应 issue 下评论征得同意；
- scheduler-plugins 在 S6 阶段 A/B 期间可替代其中一次练习（K8s 机器人多、流程规范，适合练 E2E PR 礼仪），但**不占主社区名额**。

定位纪律：这两个 PR 的目标是”跑通流程 + 让 maintainer 见过你的 ID”，diff 控制在 200 行内，不做多。

7. 阶段 3：代码级 PR（从旗舰图里长出来，issue-first 铁律）

Fluid（CP2，2027.2 前后）

来源按可信度排序：

1. **自产数据（最强故事）**：核心 1 盈亏曲面里发现的、对上游真实有用的点——候选形态：DataLoad 进度/就绪度的事件与指标暴露、预热与调度协同的设计讨论（**先在 issue 里贴你的测量数据与 proposal, major feature 按 Fluid 惯例欢迎先写设计讨论**，方向被认可后再写代码）；
2. **good-first/help-wanted 认领**：现勘期长期关注标签，选中先评论”我想试这个，思路是 X，对吗”，等点头再动手——防撞车，白嫖方案 review。

AIBrix (CP4, 2027 夏)

1. **自产需求**：核心 2 冷启动协同——以 05 的事件研究数据开 issue（“scale-out 时新副本的 TTFT 惩罚测量 + 预热/加权放量思路”），按反馈决定落地形态（routing policy 选项、benchmark harness 增强、文档化行为）；维护者可能指向已有 WIP——**被指向 WIP 不是失败，是免费情报，立即调整选题**；
2. **认领**：good first issue / help wanted 中 routing、autoscaling、benchmark 域条目，同样先评论认领。

铁律：直接甩大 PR 是新手第一大死法；feature 类一律 issue/discussion 先行；500 行内的小 PR review 周期通常是千行 PR 的零头，大改动主动拆成系列 PR。 **完成标准**：≥1 个代码级 merged PR + 完整故事复盘（发现/证据/方案分歧/最终形态/未被接受的部分及原因）。

8. issue 与 PR 模板

高质量 issue 骨架（中文/英文同构）

标题：**[组件]** 一句话现象（含关键报错或指标异常）

环境：版本 (git commit/chart 版本) / OS / kind vX / 关键配置（删敏感信息）

复现步骤（最小化，编号）：1. 2. 3.

期望行为 vs 实际行为：各一行

日志/数据：完整报错段或测量脚本+结果（代码块）

已排查项（区分新手与路人的关键）：

- 我确认过文档 X 节说 Y，与现象不符（附链接）
- 试过 A 配置，现象不变
- 怀疑与 B 有关，未验证（或：验证方法是 C，结果是 D）

反例（AI 味 issue）：无版本号；复现步骤写”按文档操作后报错”；没有已排查项；通篇礼貌空话。**每一条技术陈述必须是你亲手跑出来的**——maintainer 的第一个追问答不上来，信用即刻归零。

PR 描述结构（Fluid 用其仓库 PR 模板；AIBrix 按贡献页六要点）

1. 动机 + 关联 issue (Fixes #n / Related to #n)；
2. 改动点列表；
3. **如何验证**（命令 + 结果，maintainer 最看重的段落）；
4. 测试（新增单测/边缘用例）；
5. 文档（行为变化必须同步 docs）；
6. commit 标题 ≤50 字符、一 commit 一改动。

9. 与 AI 协作的红线（可操作版）

- **事实层自己写**：环境、复现步骤、日志、测量结果——AI 不知道你机器上发生了什么；
- **动机层自己打底**：为什么改、方案取舍先用中文写草稿，再让 AI 润成英文；
- **diff 必须自己吃透**：reviewer 有权问”这行为为什么这么写”，答不上来 = 信用破产；
- **去 AI 味**：交稿前删空洞总结句，每段要么有数据要么有步骤；保留礼貌但删掉” thanks for your great work” 式的填充堆叠；
- vLLM 系文档已有 “AI Assisted Contributions” 政策，首 PR 前去读当期版本，按其披露要求做。

10. PR 全流程工程细节（分社区，少死一次）

事项	Fluid	JuiceFS	AIbrix / Production Stack
签名	DCO git commit -s （漏签 git commit --amend -s ）	CLA 机器人 首 PR 引导；DCO 否以 CI 为准	首 PR 前核实 DCO 机器人
风格	Go + 仓库 lint	Effective Go + go fmt + pre-commit + 新文件 license header	仓库 lint/pre-commit；Go/Python 分包
commit/标题	祈使句，惯例读 10 篇 PR 总结	解释性 message，逻辑单元分开	标题 ≤50 字符；遵循仓库前缀惯例
review 纪律	1 approval； open PR ≤15	PR 必须关联 issue；大特性先设计文档	review 期间避免不必要的 force-push （官方明写，用追加 commit，merge 时由维护者处理历史）
渠道	钉钉群 7 天无 review 礼貌问一次	GitHub/Discord	Slack #AIbrix，按域 @ 维护者

通用：review 意见 **48 小时内响应**；每次 push 在 PR 留言说明改动点（reviewer 不想自己找 diff）；意见不合理就摆数据讨论，不硬顶也不无脑改；方向性分歧被否决时体面收尾（“Thanks, I see the rationale, closing this one.”）——**体面退场也是社区信用**。

11. 反模式（死亡方式清单，每周自检）

1. 不搜历史就开 issue/PR（重复提案，信用开场即负）；
2. “我先写完大 PR 再和你们说”（必关）；
3. 认领后消失、占坑不更新（超 2 周无进展主动留言）；
4. 为凑数提 typo/格式 PR（信任线里这是噪声，限 1 次走流程用）；
5. 本地测试不跑、CI 红了等机器人教（CI 红灯是给维护者发” 我需要保姆” 的信号）；
6. PR 里夹带无关重构（一个 PR 只做一件事）；

7. review 催更情绪化，或被拒后删叉走人；
8. 让贡献倒灌项目：为了 PR 改 P1/P2 的研究方向——**永远是项目产生贡献，不是贡献牵引项目**；
9. 在两个社区同时开半成品坑（精力碎片化，两条信任线都建立不起来）。

12. 风险与对策

风险	对策
issue 7 天无人理	礼貌 bump 一次；若无回应换选题，不恋战；中文渠道可在群里贴 issue 链接
good first issue 被抢	先评论认领+说思路；常备 2-3 个备选
AlBrix 迭代太快、PR 频繁冲突	rebase 纪律：每周 rebase 一次 main；冲突在 24h 内处理；选题贴近当周在改的文件反而更容易被 review
Fluid 环境/测试在本机跑不通	这本身是合法 issue；卡 3 天就在群里问，问题带版本和报错，不问”怎么入门”
review 以月计	正常，并行推进项目，不阻塞死线；月度礼貌问一次
现勘结论推翻缺口（Fluid 已做块级门控）	按 04 闸门收缩叙事：贡献改为复现数据/文档澄清，不硬推 feature
贡献吞噬项目工期	每周开源预算硬上限（见 § 13）；超限先保项目；逃生舱随时可用
法律条款变化（DCO/CLA/许可证）	每次首 PR 前以仓库当日 CONTRIBUTING/LICENSE 为准，笔记里记复核日期

13. 周节奏与本周行动清单

常态预算：使用阶段每周 ≤5 小时（stage0 期间可到 7 小时）；项目主线期每周 ≤3 小时，只做”回 issue/改 PR/读 PR”类碎片动作；自产 PR 的集中写作放在旗舰图完成后的整块周末。

本周（stage0，CP0）逐日：

1. D1: Watch fluid-cloudnative/fluid 与 vllm-project/aibrix；加入 Slack #AIBrix；fork 两仓配 upstream；
2. D2: Fluid 仓 make test /dev guide 首步，记录环境问题；
3. D3: AlBrix quickstart helm 部署 + 示例请求，截图归档；
4. D4-5: 各精读 5 篇 merged PR（四行笔记 ×10）；
5. D6: Fluid 再 5 篇 + 定向跑 P1 同域包（dataload/scheduling 相关）单测；AlBrix 跑通 routing 相关 Go 包单测；
6. D7: 写《社区地形笔记》一页（含 DCO/CLA 实际 CI 截图、响应速度初判、禁忌清单），作为 CP1 输入。

14. 诚实清单（面试主动说）

- 我的贡献是”项目驱动”的：issue 和 PR 长自 P1/P2 的现勘与测量，不是为贡献而贡献；
- typo PR 只有一个，是用来走通流程的，不包装成”活跃贡献者”；
- 被关闭/方向被否的 PR 我会保留并讲原因；
- 跨社区只有 Fluid 与 AIBrix 两条信任线，scheduler-plugins 是练习场；
- 没有碰 vLLM 引擎内核与 JuiceFS EE：语言栈/许可证边界外的事，我选择用读码和调研覆盖，不夸大。

15. v2 → v3 变更摘要

1. 主社区：JuiceFS/AIBrix 双主 → **Fluid/AIBrix 双主**，JuiceFS 降为使用基线与观察社区（缓存组是 EE，CE 无对应提 PR 空间）；
2. 指标：横跨 ≥ 2 社区 → **同社区信任线**；scheduler-plugins 明确为 S6 练习场；
3. stage0：两仓全量环境 → Fluid 全量、AIBrix quickstart 减负；20×2 篇 PR 笔记降为 10×2；
4. 事实更正：JuiceFS CE = Apache-2.0（但需签 CLA + license header）；Fluid = CNCF 沙箱/DCO/PR 模板/15 open 上限；AIBrix 维护者分工与 force-push 纪律入档；
5. 检查点与 04/05 步骤精确耦合（CP1-CP4），死线锚定 2027.2 / 2027.8；
6. 新增：社区情报档案、选题从现勘/旗舰图生长的具体候选、分社区工程细节表、反模式九条、周节奏与本周逐日清单。